STATISTICAL ANALYSIS AND FORECASTING / NETWORK ANALYSIS

Friendship Structure in an Anonymised Facebook Network

A reproducible analysis of SNAP friend network 3980 using Python

Mervine Muganguzi

6 November 2026

Abstract

This report examines observed friendships among 59 anonymised Facebook users. The graph contains 146 unique, undirected relationships and satisfies the assignment's minimum of 30 people. Five centrality measures, exact clique enumeration and Louvain community detection describe different aspects of its structure. Person 4030 has the highest degree and eigenvector centrality, while person 4023 leads closeness, betweenness and PageRank. The maximum clique size is six, with six overlapping cliques of that size covering 15 distinct people. Community boundaries vary across random seeds, so they are interpreted as algorithmic groupings. Prepared data, complete code and numerical outputs accompany the report.

1 Aim and dataset preparation

The aim is to identify well-connected people, people who connect different parts of the graph, fully connected friendship groups and broader communities. The data are from Stanford's SNAP *Social circles: Facebook* collection, originally studied by McAuley and Leskovec [1, 2].

The selected network belongs to anonymised ego user 3980. Only this user's friends and the recorded friendships *among those friends* are analysed. The ego is excluded: adding the ego would create a guaranteed connection to every friend and change the question being studied. Anonymous numerical IDs already replace personal identifiers, so names or invented initials are unnecessary. The network was selected for its friendship interpretation and manageable size; it is not a random sample of Facebook users.

Preparation procedure.

1. Read the complete roster from `3980.feat`, using only its first column of anonymised IDs. Profile feature values are not used.
2. Read `3980.edges`. Its 292 rows record each undirected friendship twice. Sort each endpoint pair and remove repeated pairs to obtain 146 unique edges. No self-loops occur.
3. Add all 59 roster nodes before adding edges. Reading only the edge file would retain 52 people and omit seven isolated nodes.
4. Save `nodes.csv` and `edges.csv`, retain source metadata and the archive's SHA-256 checksum, and use sorted inputs for reproducibility.

The graph represents an observed subnetwork. An isolate has no recorded tie to the other selected friends; this does not mean that the person has no friends elsewhere. Source circle memberships are not used; the groups below are detected from friendship edges. The archive and prepared CSV files allow the analysis to run offline. The data access date is 6 November 2026.

2 Methods and mathematical definitions

2.1 Graph notation and basic statistics

Following the lecture's graph terminology [8], let $G = (V, E)$ be a simple, unweighted, undirected graph, with $N = |V|$ people, $m = |E|$ friendships and adjacency matrix A. Thus $A_{ij} = 1$ when people i and j are friends, and zero otherwise. The degree is $k_i = \sum_j A_{ij}$. Network density and average degree are

$$\rho = \frac{2m}{N(N-1)}, \qquad \bar{k} = \frac{2m}{N}. \tag{1}$$

For vertices in the same connected component, $d(i, j)$ is the shortest-path length in number of edges. The diameter is the largest such distance. A finite whole-network diameter or average over all pairs is not reported for this disconnected graph. These distances are calculated for the largest connected component H, which contains 44 nodes.

If t_i is the number of triangles containing node i, its local clustering coefficient is $C_i = 2t_i/[k_i(k_i - 1)]$ for $k_i \geq 2$, and zero otherwise. Average clustering is $N^{-1} \sum_i C_i$. Global transitivity is $3T/\sum_i \binom{k_i}{2}$, where T is the number of triangles.

2.2 Centrality measures

All primary centralities are calculated on the full 59-node graph. A second CSV recalculates them on H with its own normalisation. The formulas follow the NetworkX definitions [4].

Degree centrality measures direct connections:

$$C_D(i) = \frac{k_i}{N-1}. \tag{2}$$

It is a measure of local connectedness; it does not by itself establish social influence.

Closeness centrality measures short-path reachability. Let R_i be the other nodes reachable from i and $r_i = |R_i|$. The Wasserman–Faust correction accounts for disconnectedness:

$$C_C(i) = \frac{r_i}{N-1} \frac{r_i}{\sum_{j \in R_i} d(i, j)}. \tag{3}$$

An isolate receives zero. Without the first factor, a node in a tiny component could appear highly central merely because the few reachable nodes are nearby.

Betweenness centrality measures shortest-path bridging. If σ_{st} is the number of shortest paths between s and t, and $\sigma_{st}(i)$ counts those passing through i, the normalised undirected measure is

$$C_B(i) = \frac{2}{(N-1)(N-2)} \sum_{\substack{s<t,\ s,t \neq i \\ \sigma_{st}>0}} \frac{\sigma_{st}(i)}{\sigma_{st}}. \tag{4}$$

The sum includes reachable pairs only; unreachable pairs contribute zero. Endpoints are excluded from path counts.

Eigenvector centrality gives more weight to ties to other central nodes:

$$Ax = \lambda_{\max} x, \qquad x_i \geq 0, \qquad \sum_i x_i^2 = 1. \tag{5}$$

For this disconnected graph, the dominant eigenvector concentrates on the 44-node component: its leading adjacency eigenvalue is approximately 9.455, compared with 3.000 for the next-largest value among the other components. Scores outside it are effectively zero and cannot usefully rank people in those smaller components.

Centrality measures (continued)

PageRank describes the stationary probability of a random walk with uniform teleportation. Each friendship is treated as two reciprocal directions. For damping factor $\alpha = 0.85$,

$$p_i = \frac{1-\alpha}{N} + \alpha \left[\sum_{j \in \mathcal{N}(i)} \frac{p_j}{k_j} + \frac{1}{N} \sum_{j:k_j=0} p_j \right], \qquad \sum_i p_i = 1. \tag{6}$$

The final term redistributes mass from isolates uniformly. Consequently every node has positive PageRank. This is an undirected friendship interpretation, not evidence of one-way popularity [5].

2.3 Cliques

A set $S \subseteq V$ is a clique when all distinct pairs are connected:

$$A_{ij} = 1 \quad \text{for every distinct } i, j \in S, \qquad \omega(G) = \max_{S \text{ is a clique}} |S|. \tag{7}$$

A *maximal* clique cannot be enlarged by adding a node. A *maximum* clique has the largest size in the entire graph. These terms are different: a small clique can be maximal without being maximum.

All cliques are enumerated to count groups by size. Maximal cliques are found with `nx.find_cliques`, which uses a Bron–Kerbosch-based algorithm; the largest of these give the exact clique number [6]. Counts are feasible for this small graph. Cliques can overlap.

2.4 Community detection and sensitivity

A community is a group with relatively concentrated internal connections; it does not require every pair to be friends. Louvain is a heuristic that seeks large modularity [3, 7]:

$$Q = \frac{1}{2m} \sum_{i,j} \left(A_{ij} - \gamma \frac{k_i k_j}{2m} \right) \mathbf{1}(c_i = c_j), \qquad \gamma = 1. \tag{8}$$

Here c_i is the community of node i. The null-model term represents expected ties under a degree-preserving model. Louvain alternates node moves that improve Q with aggregation of communities into a smaller graph. It does not guarantee the globally optimal partition or a unique social grouping.

The primary run uses seed 42. Separate partitions are fitted on the full graph and on H. They optimise different graphs, so their labels and modularity values are not directly interchangeable. On H, seeds 0–29 assess sensitivity to node processing order. The adjusted Rand index (ARI) compares each partition with seed 42 after correcting pairwise agreement for chance. If n_{uv} counts shared members of groups u and v in the two partitions, let $I = \sum_{u,v} \binom{n_{uv}}{2}$, $R = \sum_u \binom{a_u}{2}$ and $S = \sum_v \binom{b_v}{2}$, where a_u and b_v are their group sizes and $n_H = |V(H)| = 44$. Then

$$\mathrm{ARI} = \frac{I - E}{(R+S)/2 - E}, \qquad E = \frac{RS}{\binom{n_H}{2}}. \tag{9}$$

An ARI of 1 means identical assignments; values near 0 indicate chance-level agreement under fixed group sizes. Layouts also use seed 42. Display positions are not geography or numerical shortest-path distances.

3 Results: overall structure

Scope	Statistic	Value
Whole graph	People	59
	Unique friendships	146
	Density	0.0853
	Average degree	4.949
	Connected components	11
	Isolates	7
Main component	People	44
	Friendships	138
	Mean shortest-path distance	2.567
	Diameter	5

Table 1: Basic properties, with whole-network and main-component scopes separated. Distance statistics apply only to the 44-node component.

The largest component contains 44 of 59 people (74.6%) and 138 of 146 edges (94.5%). The other connected groups contain four, two and two people; seven additional components are single isolated nodes. Overall, 8.53% of possible friendship pairs are observed. Mean local clustering is 0.4069 and transitivity is 0.4500, both calculated on all 59 nodes. These describe concentration of ties around groups; they do not establish statistical significance against a random-network model. Within the

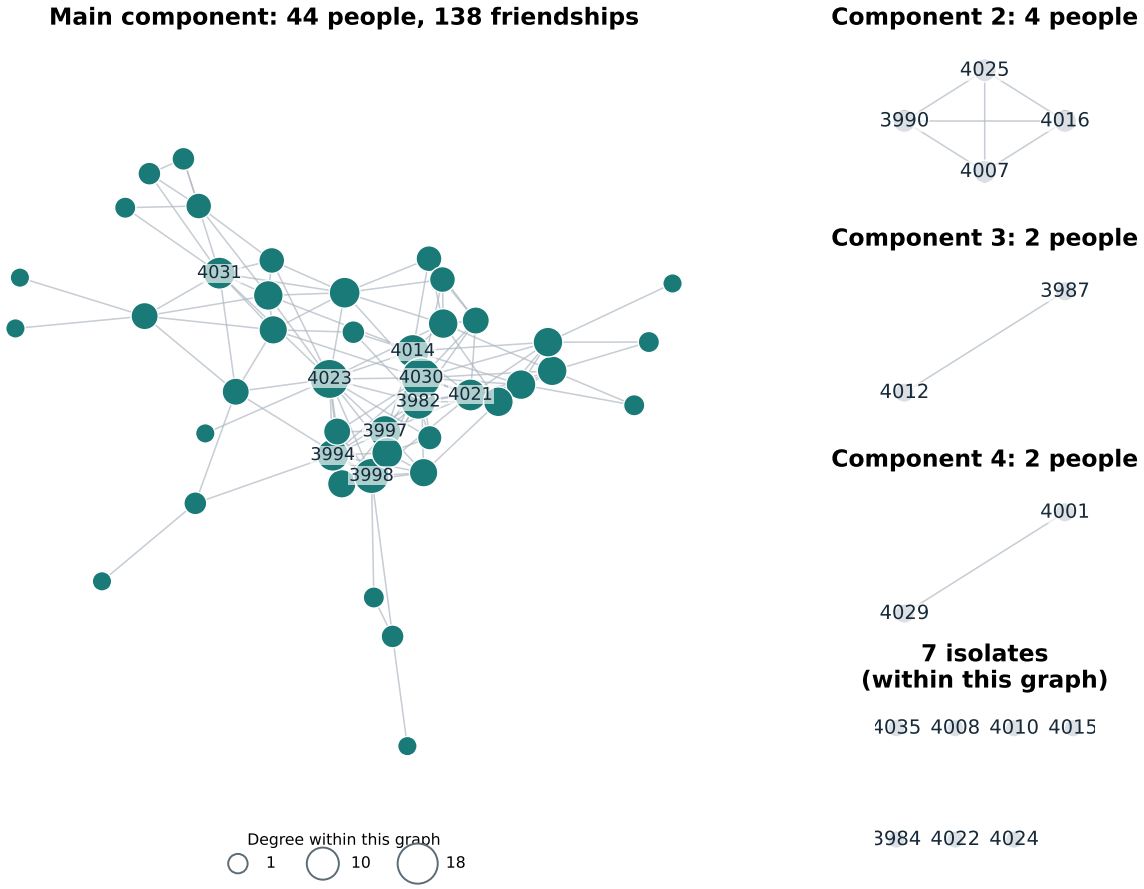


Figure 1: All 59 people, with disconnected components displayed separately. Labels are anonymised source IDs. To avoid crowding, the main panel labels people with degree at least ten or in the top five for betweenness. The size key gives degree within this graph; positions and panel separation are illustrative. Smaller components show every ID.

main component, the mean shortest path is 2.567 edges and the diameter is 5. These figures describe reachability inside this observed component, not separation across Facebook as a whole.

4 Results: centrality

ID	k	D	C	B	E	P
4030	18	0.3103	0.4035	0.1042	0.3544	0.0447
4023	17	0.2931	0.4195	0.1228	0.3211	0.0453
3982	13	0.2241	0.3707	0.0283	0.2949	0.0328
3998	13	0.2241	0.3795	0.0886	0.2634	0.0363
4014	11	0.1897	0.3795	0.0522	0.2277	0.0290
3994	10	0.1724	0.3542	0.0316	0.2427	0.0267
3997	10	0.1724	0.3503	0.0047	0.2609	0.0252
4021	10	0.1724	0.3356	0.0178	0.2262	0.0258
4031	10	0.1724	0.3391	0.0776	0.0967	0.0324
4004	9	0.1552	0.3582	0.0292	0.1427	0.0252

Table 2: Ten highest-degree people, ordered by degree then ID. k is raw degree; D, C, B, E and P denote normalised degree, corrected closeness, normalised betweenness, unit-length eigenvector centrality and PageRank. All scores use the full 59-node graph.

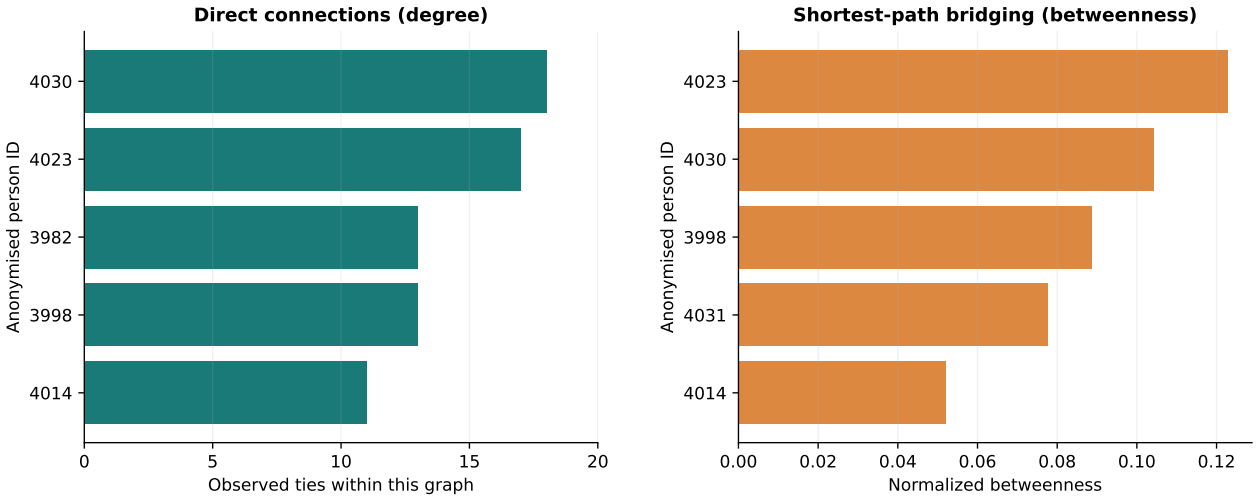


Figure 2: The top five people differ when centrality measures direct ties or shortest-path bridging. Degree ties in the top-five display are broken by ID.

Direct ties and bridging are different. Person 4030 has 18 neighbours within this selected graph and the highest degree centrality ($18/58 = 0.3103$), together with the highest eigenvector centrality. Person 4023 has 17 friends but the highest betweenness (0.1228), corrected closeness (0.4195) and PageRank (0.0453). Its betweenness rank describes its fractional contribution to shortest paths between other people in this graph.

Equal degree can hide different roles. People 3982 and 3998 both have 13 friends, but their betweenness scores are 0.0283 and 0.0886 respectively. Person 3998 therefore ranks higher as a shortest-path intermediary, with the same number of direct ties. These are structural roles; the data do not observe actual communication, influence or information flow.

The seven isolates have zero degree, closeness and betweenness. Their PageRank is positive because of teleportation. Full results for all people appear in Appendix A; a separate CSV recalculates all five measures on H.

5 Results: cliques

Size	All	Maximal
3	168	19
4	108	12
5	39	7
6	6	6

Table 3: Clique counts in the whole 59-node graph, for groups of at least three people. Counts include overlapping groups.

The graph contains 168 triangles, 108 four-person cliques and 39 five-person cliques. There are six maximum cliques, each with six people, so $\omega(G) = 6$. Each has all $\binom{6}{2} = 15$ possible friendships.

There are 44 maximal cliques of size at least three. The six maximum cliques overlap and together cover 15 distinct people. For example, the first two maximum cliques below share five people.

Clique	Anonymised person IDs
1	3981, 3994, 3997, 3998, 4009, 4019
2	3981, 3994, 3997, 3998, 4009, 4023
3	3981, 3994, 3997, 3998, 4018, 4023
4	3982, 3994, 3997, 3998, 4009, 4023
5	3982, 3994, 3997, 4009, 4023, 4030
6	3986, 4000, 4017, 4021, 4026, 4030

Table 4: All six maximum cliques, in reproducible lexicographic order.

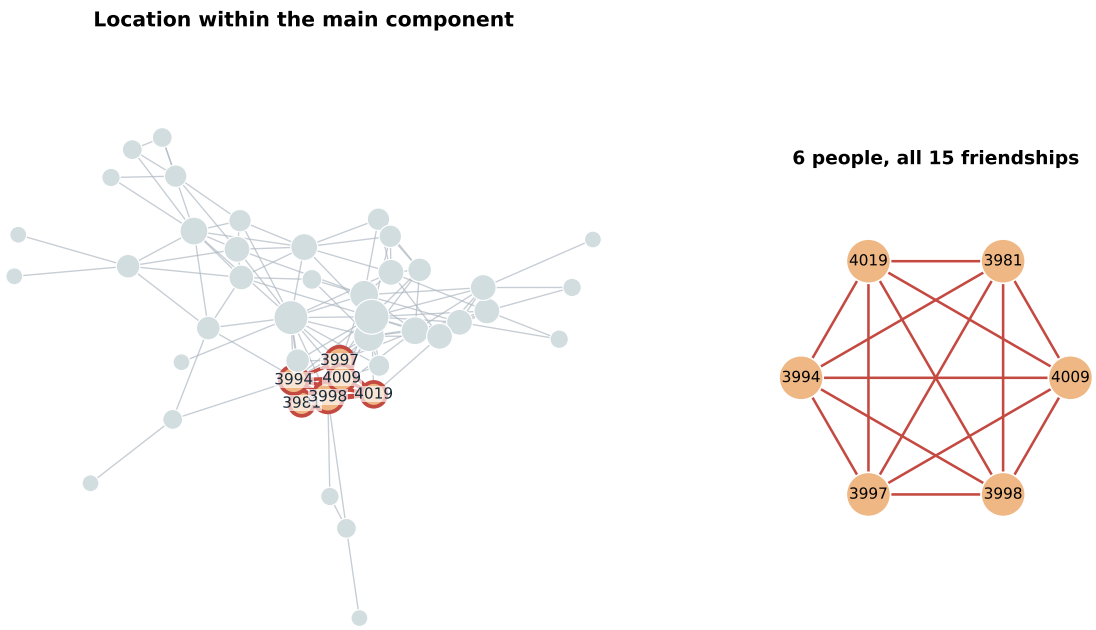


Figure 3: One maximum clique. Left: its location highlighted within the main component. Right: a separate circular view shows all 15 friendships among its six people. These people can also have ties outside the clique.

6 Results: communities

On the full graph, Louvain with seed 42 returns 14 groups and $Q = 0.4576$. Seven of these groups are individual isolates. An isolate has degree zero, so its community assignment does not change modularity; Louvain leaves it as a singleton. Other groups reflect disconnected components. Thus the total count of 14 is an algorithm output, not an estimate of 14 real social circles. To examine finer structure, Louvain is fitted again on the main component alone. The four groups contain 16, 14, 11

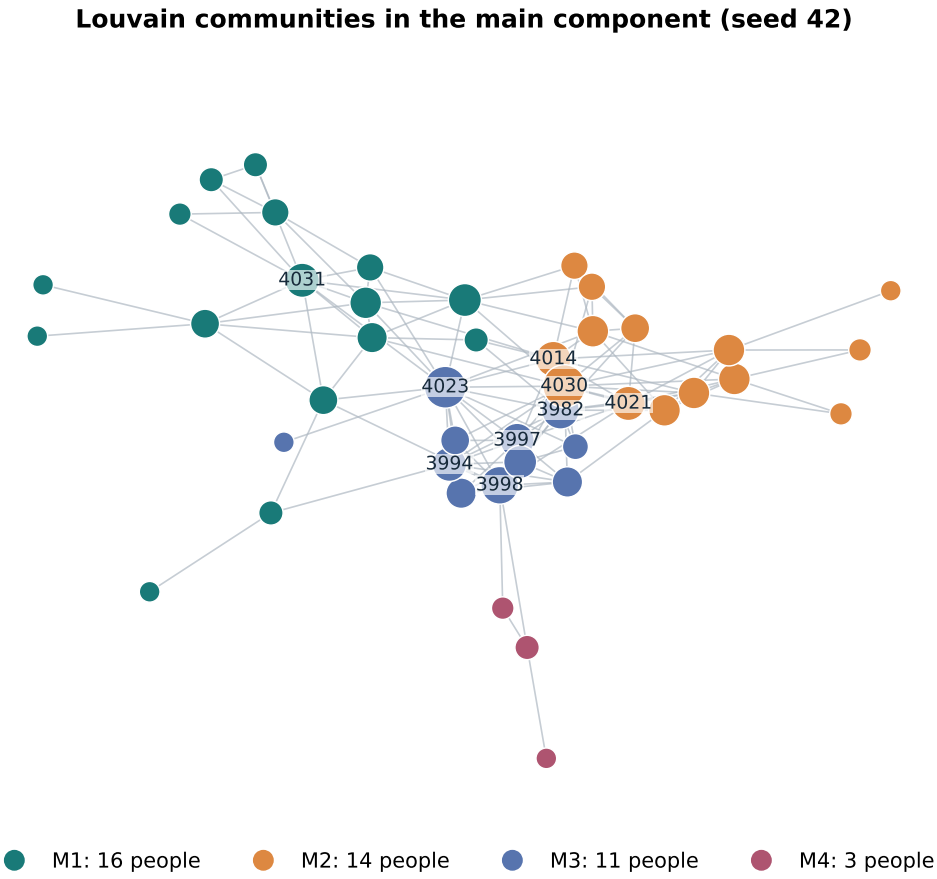


Figure 4: A four-community partition of the main 44-person component, using seed 42 and resolution 1. Sizes use the degree key in Figure 1; only the key people identified there are labelled. Every person's community is listed in Appendix A. Colours carry no inferred family, school or workplace meaning.

Group	People	Internal ties	External ties	Density
M1	16	29	15	0.242
M2	14	37	26	0.407
M3	11	34	29	0.618
M4	3	2	2	0.667

Table 5: Main-component communities. External edges count ties from that group to other groups, so a between-group edge is counted once for each endpoint group.

and 3 people, with $Q = 0.4139$. Of 138 main- component edges, 102 are inside groups and 36 connect different groups. M3 is a relatively dense larger group (34 of 55 possible internal ties). M4 contains only two of three possible ties: it is a community in this partition but is not a three-person clique. Density alone is sensitive to group size.

7 Sensitivity, limitations and conclusions

7.1 How stable are the community results?

Thirty additional Louvain runs on H, using seeds 0–29 with the same resolution, produce four communities in 11 runs and five in 19 runs. Their modularity values range from 0.4064 to 0.4151. Agreement with the seed-42 partition ranges from ARI = 0.441 to 1.000, showing that some membership boundaries also change. Thus, the four-colour figure is one reproducible, plausible partition. It is not a claim that exactly four objectively defined social groups exist. The seed-42 result was fixed for reproducibility; it was not selected as the highest-Q run.

7.2 Limits of interpretation

- **Scope and selection.** This is one person's observed friend network. The focal person is excluded and ties outside the selected group are unobserved. The findings cannot be generalised to all Facebook users or the student's own friends.
- **Anonymisation.** Anonymous IDs preserve source identity across tables but cannot support labels such as family, classmates or colleagues. No personal profile features are used to explain the communities.
- **Disconnectedness.** The full graph has 11 components. Closeness uses an explicit reachability correction. Eigenvector scores outside the dominant component are effectively zero; global path statistics would be misleading.
- **Static, binary ties.** Edges indicate recorded friendship, not its strength, frequency or direction. There are no repeated time observations here, so no forecasting model or claim about current friendships is justified.
- **Algorithmic choices and validation.** Louvain depends on random order and resolution. Modularity is a descriptive objective, not a significance test. The source includes ego-defined circle memberships, but this report does not compare the detected communities with that reference. Their agreement has therefore not been assessed. Anonymous circle names do not identify social meaning.

7.3 Conclusions

Within the selected graph, person 4030 has the most direct ties and person 4023 ranks highest on betweenness. Six maximum six-person cliques overlap across 15 distinct people. Louvain yields four or five groups inside the main connected component across the tested seeds. The network therefore combines several disconnected components, tightly connected local groups and people with distinct structural roles; no one centrality ranking captures all of those properties.

8 Reproducing the analysis

The accompanying project contains the full data preparation, calculations, visualisations and exported tables. It was run with Python 3.12.14, NetworkX 3.7, Matplotlib 3.10.8, NumPy 2.3.5 and SciPy 1.17.0. The exact dependency versions are in `requirements.txt`. From the extracted project directory:

```
python -m pip install -r requirements.txt
python network_analysis.py
```

The bundled archive permits this second command to run offline. The script recreates prepared CSVs in `data/`, numerical outputs in `results/` and vector PDF plus PNG figures in `figures/`. Tables retain more decimal places than the rounded report. `run_analysis.ipynb` provides a notebook entry point. Rebuilding this report additionally requires a LaTeX installation; instructions are in `README.txt`.

Validation includes the handshaking identity $\sum_i k_i = 2m$, triangle counts from two algorithms, pairwise completeness of enumerated maximal cliques, and PageRank probabilities summing to one. The optional `verify_results.py` independently checks the saved outputs using direct shortest-path enumeration, matrix calculations and recursive clique enumeration. It saves `results/verification.json`. Complete analysis and plotting code appears in Appendix B.

References

[1] Stanford Network Analysis Project. *Social circles: Facebook*. Dataset and file-format documentation. Accessed 6 November 2026. https://snap.stanford.edu/data/ego-Facebook.html and https://snap.stanford.edu/data/readme-Ego.txt.

[2] McAuley, J. and Leskovec, J. (2012). Learning to Discover Social Circles in Ego Networks. *Advances in Neural Information Processing Systems*, 25.

[3] Blondel, V. D., Guillaume, J.-L., Lambiotte, R. and Lefebvre, E. (2008). Fast unfolding of communities in large networks. *Journal of Statistical Mechanics*, P10008. https://doi.org/10.1088/1742-5468/2008/10/P10008.

[4] NetworkX Developers. Centrality documentation, including normalisation and disconnected-graph conventions. https://networkx.org/documentation/stable/reference/algorithms/centrality.html.

[5] NetworkX Developers. `pagerank` documentation. https://networkx.org/documentation/stable/reference/algorithms/generated/networkx.algorithms.link_analysis.pagerank_alg.pagerank.html.

[6] NetworkX Developers. `find_cliques` documentation. https://networkx.org/documentation/stable/reference/algorithms/generated/networkx.algorithms.clique.find_cliques.html.

[7] NetworkX Developers. `louvain_communities` documentation. https://networkx.org/documentation/stable/reference/algorithms/generated/networkx.algorithms.community.louvain.louvain_communities.html.

[8] Kavaliauskas, M. (2025). *Graph and network analysis*. KTU lecture slides, 28 October 2025. Supplied with the assignment.

A Centrality values for all 59 people

Values are rounded to four decimals and use the full graph. Comp. denotes connected component. F denotes a community fitted on all 59 people; M denotes a community fitted separately on the 44-node main component, both using seed 42. A dash means the person is outside that main component. Components and community labels are numbered by decreasing size, with minimum ID breaking ties. The CSV files preserve full precision. A printed zero for eigenvector centrality can represent a very small numerical value outside the dominant component.

ID	k	D	C	B	E	P	Comp.	F	M
594	3	0.0517	0.2397	0.0003	0.0142	0.0123	1	1	1
3981	7	0.1207	0.3065	0.0009	0.1771	0.0186	1	2	3
3982	13	0.2241	0.3707	0.0283	0.2949	0.0328	1	2	3
3983	1	0.0172	0.2214	0.0000	0.0148	0.0054	1	3	2
3984	0	0.0000	0.0000	0.0000	0.0000	0.0028	5	8	–
3985	5	0.0862	0.2872	0.0014	0.0760	0.0145	1	4	2
3986	8	0.1379	0.3125	0.0161	0.1672	0.0221	1	3	2
3987	1	0.0172	0.0172	0.0000	0.0000	0.0189	3	6	–
3988	6	0.1034	0.2925	0.0030	0.1243	0.0165	1	4	2
3989	5	0.0862	0.2510	0.0044	0.0235	0.0188	1	1	1
3990	3	0.0517	0.0517	0.0000	0.0000	0.0189	2	5	–
3991	2	0.0345	0.2379	0.0003	0.0127	0.0088	1	1	1
3992	2	0.0345	0.2261	0.0000	0.0289	0.0079	1	3	2
3993	5	0.0862	0.2952	0.0014	0.0881	0.0144	1	4	2
3994	10	0.1724	0.3542	0.0316	0.2427	0.0267	1	2	3
3995	8	0.1379	0.3356	0.0153	0.1360	0.0217	1	4	2
3996	3	0.0517	0.2725	0.0259	0.0356	0.0136	1	1	1
3997	10	0.1724	0.3503	0.0047	0.2609	0.0252	1	2	3
3998	13	0.2241	0.3795	0.0886	0.2634	0.0363	1	2	3
3999	3	0.0517	0.2592	0.0254	0.0315	0.0154	1	2	4
4000	8	0.1379	0.3036	0.0209	0.1330	0.0232	1	3	2
4001	1	0.0172	0.0172	0.0000	0.0000	0.0189	4	7	–
4002	6	0.1034	0.3356	0.0427	0.0903	0.0202	1	1	1
4003	4	0.0690	0.3125	0.0000	0.1275	0.0115	1	2	3
4004	9	0.1552	0.3582	0.0292	0.1427	0.0252	1	4	1
4005	2	0.0345	0.2571	0.0000	0.0312	0.0096	1	2	4
4006	1	0.0172	0.2111	0.0000	0.0045	0.0063	1	1	1
4007	3	0.0517	0.0517	0.0000	0.0000	0.0189	2	5	–
4008	0	0.0000	0.0000	0.0000	0.0000	0.0028	6	9	–
4009	9	0.1552	0.3465	0.0041	0.2351	0.0231	1	2	3
4010	0	0.0000	0.0000	0.0000	0.0000	0.0028	7	10	–
4011	3	0.0517	0.2397	0.0003	0.0142	0.0123	1	1	1
4012	1	0.0172	0.0172	0.0000	0.0000	0.0189	3	6	–
4013	5	0.0862	0.3065	0.0061	0.0732	0.0160	1	1	1
4014	11	0.1897	0.3795	0.0522	0.2277	0.0290	1	3	2
4015	0	0.0000	0.0000	0.0000	0.0000	0.0028	8	11	–
4016	3	0.0517	0.0517	0.0000	0.0000	0.0189	2	5	–
4017	8	0.1379	0.3125	0.0392	0.1401	0.0244	1	3	2
4018	6	0.1034	0.3321	0.0010	0.1713	0.0162	1	2	3
4019	7	0.1207	0.3095	0.0038	0.1805	0.0184	1	2	3
4020	7	0.1207	0.3582	0.0338	0.0952	0.0218	1	1	1
4021	10	0.1724	0.3356	0.0178	0.2262	0.0258	1	3	2
4022	0	0.0000	0.0000	0.0000	0.0000	0.0028	9	12	–
4023	17	0.2931	0.4195	0.1228	0.3211	0.0453	1	2	3
4024	0	0.0000	0.0000	0.0000	0.0000	0.0028	10	13	–
4025	3	0.0517	0.0517	0.0000	0.0000	0.0189	2	5	–
4026	8	0.1379	0.3095	0.0047	0.1726	0.0212	1	3	2
4027	6	0.1034	0.2925	0.0508	0.0422	0.0244	1	1	1
4028	1	0.0172	0.2005	0.0000	0.0038	0.0067	1	1	1
4029	1	0.0172	0.0172	0.0000	0.0000	0.0189	4	7	–
4030	18	0.3103	0.4035	0.1042	0.3544	0.0447	1	3	2
4031	10	0.1724	0.3391	0.0776	0.0967	0.0324	1	1	1
4032	1	0.0172	0.2111	0.0000	0.0045	0.0063	1	1	1
4033	2	0.0345	0.2261	0.0000	0.0317	0.0076	1	3	2
4034	1	0.0172	0.2702	0.0000	0.0340	0.0051	1	2	3
4035	0	0.0000	0.0000	0.0000	0.0000	0.0028	11	14	–
4036	1	0.0172	0.1932	0.0000	0.0033	0.0072	1	2	4
4037	3	0.0517	0.3007	0.0008	0.0653	0.0099	1	1	1
4038	8	0.1379	0.3465	0.0368	0.1081	0.0245	1	1	1

B Complete Python analysis and plotting code

The following is the entire `network_analysis.py` file used for the report. It prepares the data, computes both graph scopes, enumerates cliques, runs community sensitivity analysis, writes numerical outputs and creates every plot. Report typesetting files are included separately in the project bundle.

```
"""Analyse SNAP Facebook friend network 3980 (Python 3.12+).

Prepared for: Mervine Muganguzi

Run: python network_analysis.py
The bundled archive permits an offline rerun. All paths are relative to this file.
"""
from collections import Counter
from importlib.metadata import version
from itertools import combinations
from math import comb
from pathlib import Path
import csv
import hashlib
import json
import platform
import tarfile
from urllib.request import urlopen

import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
from matplotlib.lines import Line2D
import networkx as nx

ROOT = Path(__file__).resolve().parent
DATA, OUT, FIG = [ROOT / x for x in ("data", "results", "figures")]
SOURCE = "https://snap.stanford.edu/data/facebook.tar.gz"
EGO, SEED = 3980, 42
COLORS = ["#197A78", "#DD8841", "#5774AE", "#AE5470", "#8A9545"]
plt.rcParams.update({"font.family": "DejaVu Sans", "font.size": 10,
                     "axes.spines.top": False, "axes.spines.right": False,
                     "axes.titleweight": "bold", "savefig.facecolor": "white"})


def write_csv(name, rows, fields):
    """Write a table with an explicit column order."""
    with (OUT / name).open("w", newline="", encoding="utf-8") as stream:
        writer = csv.DictWriter(stream, fieldnames=fields)
        writer.writeheader()
        writer.writerows(rows)


def partition(graph, seed=SEED):
    """Sort Louvain communities so their displayed labels are reproducible."""
    groups = nx.community.louvain_communities(
        graph, weight=None, resolution=1, seed=seed)
    return sorted(groups, key=lambda c: (-len(c), min(c)))


def membership(groups):
    return {node: i + 1 for i, group in enumerate(groups) for node in group}


def adjusted_rand(a, b, nodes):
    """Chance-adjusted agreement of two community assignments."""
    if len(nodes) < 2:
        return 1.0
    pairs = lambda n: comb(n, 2)
    cells = Counter((a[v], b[v]) for v in nodes)
    score = sum(pairs(n) for n in cells.values())
    row = sum(pairs(n) for n in Counter(a[v] for v in nodes).values())
    col = sum(pairs(n) for n in Counter(b[v] for v in nodes).values())
    expected = row * col / pairs(len(nodes))
    limit = (row + col) / 2
    return (score - expected) / (limit - expected) if limit != expected else 1.0


def centralities(graph):
    """Calculate the five measures for the supplied graph and its scope."""
    return {
```

```
        "degree_centrality": nx.degree_centrality(graph),
        "closeness": nx.closeness_centrality(graph, wf_improved=True),
        "betweenness": nx.betweenness_centrality(graph, normalized=True),
        "eigenvector": nx.eigenvector_centrality(
            graph, max_iter=5000, tol=1e-12),
        "pagerank": nx.pagerank(graph, alpha=0.85, max_iter=5000, tol=1e-12),
    }


def save_figure(fig, name):
    fig.savefig(FIG / (name + ".pdf"), bbox_inches="tight")
    fig.savefig(FIG / (name + ".png"), dpi=210, bbox_inches="tight")
    plt.close(fig)


def plot_network(graph, pos, ax, color, title, highlighted=None, label_nodes=None):
    """Use a fixed size key; selectively label crowded plots for readability."""
    highlight = set() if highlighted is None else set(highlighted)
    nx.draw_networkx_edges(graph, pos, ax=ax, edge_color="#AEB8C2",
                           width=0.8, alpha=0.7)
    if highlight:
        internal = [(u, v) for u, v in graph.edges if u in highlight and v in highlight]
        nx.draw_networkx_edges(graph, pos, ax=ax, edgelist=internal,
                               edge_color="#C44941", width=2.1)
    nx.draw_networkx_nodes(
        graph, pos, ax=ax, node_size=[80 + 19 * graph.degree(v) for v in graph],
        node_color=color, edgecolors=["#C44941" if v in highlight else "white"
                                      for v in graph],
        linewidths=[2.2 if v in highlight else 0.7 for v in graph])
    labelled = graph if label_nodes is None else label_nodes
    labels = {v: str(v) for v in sorted(labelled) if v in graph}
    nx.draw_networkx_labels(graph, pos, labels=labels, ax=ax,
                            font_size=9 if len(graph) > 10 else 10,
                            font_color="#172A3A",
                            bbox={"facecolor": "white", "edgecolor": "none",
                                  "alpha": 0.65, "pad": 0.25})
    ax.set_title(title, fontsize=12, pad=14)
    ax.margins(0.12)
    ax.axis("off")


def size_key(ax):
    handles = [ax.scatter([], [], s=80 + 19 * degree, facecolor="white",
                          edgecolors="#5C6C77", label=str(degree))
               for degree in (1, 10, 18)]
    ax.legend(handles=handles, title="Degree within this graph",
              loc="lower center", bbox_to_anchor=(0.5, -0.04),
              ncol=3, frameon=False, fontsize=8, title_fontsize=8)


def main():
    for folder in (DATA, OUT, FIG):
        folder.mkdir(exist_ok=True)
    archive = DATA / "facebook.tar.gz"
    if not archive.exists():
        with urlopen(SOURCE, timeout=60) as response:
            archive.write_bytes(response.read())
    archive_hash = hashlib.sha256(archive.read_bytes()).hexdigest()
    expected_hash = "78594c43169b698e99a77741bc298581af87146e4f3d3e8ad48a48361b5d92b6"
    if archive_hash != expected_hash:
        raise ValueError("The source archive differs from the audited snapshot.")
    # Read specific files in memory; never extract arbitrary archive paths.
    with tarfile.open(archive) as tar:
        files = {Path(m.name).name: m for m in tar if m.isfile()}
        edge_text = tar.extractfile(files[f"{EGO}.edges"]).read().decode()
        feat_text = tar.extractfile(files[f"{EGO}.feat"]).read().decode()
    nodes = sorted({int(line.split()[0]) for line in feat_text.splitlines() if line})
    raw_edges = [tuple(map(int, line.split())) for line in edge_text.splitlines()
                 if line.strip()]
    edges = sorted({tuple(sorted(edge)) for edge in raw_edges})
    assert set(Counter(tuple(sorted(e)) for e in raw_edges).values()) == {2}
    assert all(u != v for u, v in edges), "Unexpected self-loop in source"
    assert {v for edge in edges for v in edge} <= set(nodes)
    assert EGO not in nodes
    graph = nx.Graph()
    graph.add_nodes_from(nodes)  # Preserve seven isolates absent from .edges.
    graph.add_edges_from(edges)
    assert (len(graph), graph.number_of_edges()) == (59, 146)
    for name, fields, rows in [
```

```
        ("nodes.csv", ["node_id"], [[v] for v in nodes]),
        ("edges.csv", ["source", "target"], edges),
    ]:
        with (DATA / name).open("w", newline="") as stream:
            writer = csv.writer(stream)
            writer.writerow(fields)
            writer.writerows(rows)
    provenance = {
        "dataset": "SNAP Social circles: Facebook, friend network 3980",
        "source_page": "https://snap.stanford.edu/data/ego-Facebook.html",
        "archive_url": SOURCE,
        "source_documentation": "https://snap.stanford.edu/data/readme-Ego.txt",
        "citation": "J. McAuley and J. Leskovec (2012), Learning to Discover "
                    "Social Circles in Ego Networks, NIPS 25.",
        "accessed_date": "2026-11-06",
        "archive_sha256": archive_hash,
        "scope": "Friends of ego 3980; ego excluded; profile values not used",
        "unused_source_fields": ["profile feature values", "ego-defined circle memberships"],
        "roster_file": "3980.feat", "relationship_file": "3980.edges",
        "raw_edge_rows": len(raw_edges), "unique_undirected_edges": len(edges),
        "node_count": len(nodes), "anonymized_original_ids_retained": True,
    }
    (DATA / "source.json").write_text(json.dumps(provenance, indent=2))
    components = sorted(nx.connected_components(graph),
                        key=lambda c: (-len(c), min(c)))
    component_id = membership(components)
    main_graph = graph.subgraph(components[0]).copy()
    full_groups, main_groups = partition(graph), partition(main_graph)
    full_labels, main_labels = membership(full_groups), membership(main_groups)
    full_scores, main_scores = centralities(graph), centralities(main_graph)
    fields = ["node_id", "degree", "degree_centrality", "closeness",
              "betweenness", "eigenvector", "pagerank", "component",
              "full_community", "main_community"]
    for g, scores, filename in [(graph, full_scores, "centrality_all_nodes.csv"),
                                (main_graph, main_scores, "centrality_main_component.csv")]:
        rows = [{"node_id": v, "degree": g.degree(v),
                 **{key: value[v] for key, value in scores.items()},
                 "component": component_id[v], "full_community": full_labels[v],
                 "main_community": main_labels.get(v, "")} for v in sorted(g)]
        write_csv(filename, rows, fields)
    maximal = sorted((sorted(c) for c in nx.find_cliques(graph)),
                     key=lambda c: (-len(c), c))
    clique_counts = Counter(len(c) for c in nx.enumerate_all_cliques(graph))
    maximum_size = max(map(len, maximal))
    largest = [c for c in maximal if len(c) == maximum_size]
    assert all(graph.has_edge(u, v) for c in maximal for u, v in combinations(c, 2))
    write_csv("maximal_cliques.csv", [{"clique_id": i + 1, "size": len(c),
              "node_ids": " ".join(map(str, c))} for i, c in enumerate(maximal)],
              ["clique_id", "size", "node_ids"])
    write_csv("clique_counts.csv", [{"size": k, "all_cliques": clique_counts[k],
              "maximal_cliques": sum(len(c) == k for c in maximal)}
              for k in range(1, maximum_size + 1)],
              ["size", "all_cliques", "maximal_cliques"])
    community_stats = []
    for scope, g, groups in [("full", graph, full_groups),
                             ("main", main_graph, main_groups)]:
        for i, group in enumerate(groups, 1):
            internal = g.subgraph(group).number_of_edges()
            external = sum(1 for u, v in g.edges if (u in group) != (v in group))
            community_stats.append({"scope": scope, "community": i,
                "nodes": len(group), "internal_edges": internal,
                "external_edges": external, "density": nx.density(g.subgraph(group)),
                "node_ids": " ".join(map(str, sorted(group)))})
    write_csv("communities.csv", community_stats, list(community_stats[0]))
    sensitivity = []
    for seed in range(30):
        groups = partition(main_graph, seed)
        sensitivity.append({"seed": seed, "communities": len(groups),
            "modularity": nx.community.modularity(main_graph, groups, weight=None),
            "adjusted_rand_vs_seed42": adjusted_rand(
                membership(groups), main_labels, list(main_graph))})
    write_csv("community_sensitivity.csv", sensitivity, list(sensitivity[0]))
    summary = {
        "nodes": len(graph), "edges": len(edges), "density": nx.density(graph),
        "raw_edge_rows": len(raw_edges), "duplicate_edge_rows_removed": len(raw_edges)-len(edges),
        "average_degree": 2 * len(edges) / len(graph),
        "degree_range": [min(dict(graph.degree()).values()), max(dict(graph.degree()).values())],
        "component_sizes": [len(c) for c in components],
        "isolates": sorted(nx.isolates(graph)),
```

```
        "main_nodes": len(main_graph), "main_edges": main_graph.number_of_edges(),
        "main_diameter": nx.diameter(main_graph),
        "main_average_distance": nx.average_shortest_path_length(main_graph),
        "average_clustering": nx.average_clustering(graph),
        "transitivity": nx.transitivity(graph), "triangles": clique_counts[3],
        "maximum_clique_size": maximum_size, "maximum_cliques": largest,
        "people_in_maximum_cliques": len(set().union(*map(set, largest))),
        "maximal_cliques_size3plus": sum(len(c) >= 3 for c in maximal),
        "full_community_sizes": [len(c) for c in full_groups],
        "full_modularity": nx.community.modularity(graph, full_groups, weight=None),
        "main_community_sizes": [len(c) for c in main_groups],
        "main_modularity": nx.community.modularity(main_graph, main_groups, weight=None),
        "seed": SEED, "resolution": 1, "sensitivity_seeds": [0, 29],
        "sensitivity_counts": dict(Counter(r["communities"] for r in sensitivity)),
        "sensitivity_modularity_range": [min(r["modularity"] for r in sensitivity),
                                         max(r["modularity"] for r in sensitivity)],
        "sensitivity_ari_range": [min(r["adjusted_rand_vs_seed42"] for r in sensitivity),
                                  max(r["adjusted_rand_vs_seed42"] for r in sensitivity)],
        "top5": {key: sorted(values, key=lambda v: (-values[v], v))[:5]
                 for key, values in full_scores.items()},
        "versions": {p: version(p) for p in ("networkx", "matplotlib", "numpy", "scipy")},
        "python": platform.python_version(),
    }
    assert abs(sum(full_scores["pagerank"].values()) - 1) < 1e-9
    assert sum(dict(graph.degree()).values()) == 2 * len(edges)
    assert clique_counts[3] == sum(nx.triangles(graph).values()) // 3
    (OUT / "summary.json").write_text(json.dumps(summary, indent=2))
    # Use the same main-component layout in all network figures.
    main_pos = nx.spring_layout(main_graph, seed=SEED, iterations=500, k=0.48)
    key_nodes = set(summary["top5"]["degree_centrality"])
    key_nodes.update(summary["top5"]["betweenness"])
    key_nodes.update(v for v in graph if graph.degree(v) >= 10)
    # Separate panels express disconnectedness, not geographic separation.
    fig = plt.figure(figsize=(11.0, 7.4))
    grid = fig.add_gridspec(4, 4, width_ratios=[1, 1, 1, 0.9])
    ax = fig.add_subplot(grid[:, :3])
    plot_network(main_graph, main_pos, ax, COLORS[0],
                 "Main component: 44 people, 138 friendships", label_nodes=key_nodes)
    size_key(ax)
    for i, component in enumerate(components[1:4]):
        small = graph.subgraph(component)
        ax = fig.add_subplot(grid[i, 3])
        plot_network(small, nx.circular_layout(small), ax, "#9CA7B2",
                     f"Component {i + 2}: {len(small)} people")
    isolated = graph.subgraph(summary["isolates"])
    ax = fig.add_subplot(grid[3, 3])
    pos = {v: (i % 4, -(i // 4)) for i, v in enumerate(isolated)}
    plot_network(isolated, pos, ax, "#9CA7B2", "7 isolates\n(within this graph)")
    fig.subplots_adjust(wspace=0.35, hspace=0.6)
    save_figure(fig, "01_network_overview")
    fig, ax = plt.subplots(figsize=(10, 7.0))
    plot_network(main_graph, main_pos, ax,
                 [COLORS[main_labels[v] - 1] for v in main_graph],
                 "Louvain communities in the main component (seed 42)",
                 label_nodes=key_nodes)
    handles = [Line2D([], [], marker="o", linestyle="", color=COLORS[i],
               label=f"M{i+1}: {len(c)} people", markersize=9)
               for i, c in enumerate(main_groups)]
    ax.legend(handles=handles, loc="lower center", ncol=4, frameon=False,
              bbox_to_anchor=(0.5, -0.03))
    save_figure(fig, "02_communities")
    fig, axes = plt.subplots(1, 2, figsize=(10.5, 4.3))
    for ax, key, title in zip(axes, ["degree_centrality", "betweenness"],
                              ["Direct connections (degree)", "Shortest-path bridging (betweenness)"]):
        selected = summary["top5"][key][::-1]
        values = [graph.degree(v) if key == "degree_centrality" else full_scores[key][v]
                  for v in selected]
        ax.barh([str(v) for v in selected], values, color=COLORS[0 if key == "degree_centrality" else 1])
        ax.set_title(title, fontsize=11)
        ax.set_xlabel("Observed ties within this graph" if key == "degree_centrality"
                      else "Normalized betweenness")
        ax.set_ylabel("Anonymised person ID")
        if key == "degree_centrality":
            upper = ((max(values) + 4) // 5) * 5
            ax.set_xticks(range(0, upper + 1, 5))
            ax.set_xlim(0, upper)
        ax.grid(axis="x", alpha=0.18)
    fig.tight_layout(w_pad=3)
    save_figure(fig, "03_centrality_comparison")
```

```
    fig, axes = plt.subplots(1, 2, figsize=(10.5, 6.0),
                             gridspec_kw={"width_ratios": [1.8, 1]})
    plot_network(main_graph, main_pos, axes[0],
                 ["#EEB784" if v in largest[0] else "#D2DDE0" for v in main_graph],
                 "Location within the main component", largest[0], largest[0])
    clique_graph = graph.subgraph(largest[0])
    circle = nx.circular_layout(clique_graph)
    nx.draw_networkx_edges(clique_graph, circle, ax=axes[1], width=1.5,
                           edge_color="#C44941")
    nx.draw_networkx_nodes(clique_graph, circle, ax=axes[1], node_size=680,
                           node_color="#EEB784", edgecolors="white")
    nx.draw_networkx_labels(clique_graph, circle, ax=axes[1], font_size=9)
    axes[1].set_title("6 people, all 15 friendships", fontsize=11, pad=14)
    axes[1].margins(0.24)
    axes[1].set_aspect("equal")
    axes[1].axis("off")
    fig.tight_layout()
    save_figure(fig, "04_maximum_clique")
    fig, axes = plt.subplots(1, 2, figsize=(10.5, 3.5))
    degrees = Counter(dict(graph.degree()).values())
    axes[0].bar(list(degrees), list(degrees.values()), color=COLORS[0])
    axes[0].set(xlabel="Degree (number of observed friends)", ylabel="People",
                title="Degree distribution", xticks=range(0, 19, 2))
    sizes = list(range(3, maximum_size + 1))
    axes[1].bar(sizes, [clique_counts[k] for k in sizes], color=COLORS[1])
    axes[1].set(xlabel="Clique size", ylabel="Number of cliques",
                title="All cliques of size 3 or more", xticks=sizes)
    fig.tight_layout(w_pad=3)
    save_figure(fig, "05_distributions")
    print(json.dumps(summary, indent=2))
    print("Finished: prepared data in data/, tables in results/, plots in figures/.")


if __name__ == "__main__":
    main()
```